

Reporte Técnico: Plan para operar bien con 200 usuarios concurrentes

Proyecto: Druvle Stock | Fecha: 11 de mayo de 2026 | Versión: 1.0

1. Resumen ejecutivo

La base de arquitectura actual (Laravel monolítico modular con MVC, servicios/repositorios, eventos y colas) es válida para llegar a 200 usuarios concurrentes. El principal trabajo pendiente está en la capa operativa: caché/sesión/colas, escalado horizontal, tuning de PHP-FPM y base de datos, y observabilidad.

Conclusión: Con las acciones de este plan, el sistema debería operar de forma estable con 200 usuarios concurrentes y dejar preparado el camino para escalar a 500+.

2. Estado actual y brechas

2.1 Fortalezas actuales

- Separación funcional por capas (controladores, servicios, repositorios).
- Eventos y listeners en cola para notificaciones.
- Multi-tenant con scope global y middleware dedicado.
- Docker y Nginx ya integrados.

2.2 Brechas para 200 concurrentes

- Cola en base de datos (correcta para inicio, menos eficiente en picos).
- Caché por archivos y sesiones en DB sin Redis compartido.
- Solo una instancia app y un worker en docker-compose.
- Sin observabilidad avanzada (APM, métricas p95, alertas operativas).
- Sin política formal de pruebas de carga recurrentes.

3. Objetivos operativos (SLO) para declarar éxito

- **Latencia web p95:** menor a 500 ms en endpoints críticos.
- **Error rate:** menor a 1% en 5xx durante carga.
- **Cola:** espera promedio menor a 5 s en notificaciones.
- **Recursos:** CPU media menor a 70% sostenida en app y DB.
- **Disponibilidad:** mayor o igual a 99.5% mensual.

4. Plan de evolución por fases

Fase A (Base de rendimiento, 1-2 semanas)

Acción	Qué hacer	Resultado esperado
Migrar a Redis	Usar Redis para <i>cache</i> , <i>session</i> y <i>queue</i> .	Menor I/O a disco/DB y mejor rendimiento concurrente.
Optimizar PHP-FPM	Ajustar <i>pm</i> , <i>pm.max_children</i> , <i>opcache</i> .	Mejor throughput y menor latencia en picos.
Optimizar workers	Levantar 2-4 workers con políticas de reintento y timeout.	Cola estable y sin acumulación prolongada.
Índices SQL	Indexar columnas de filtro/joins: <i>tenant_id</i> , fechas, estado, <i>user_id</i> .	Consultas de lectura más rápidas y predecibles.

Fase B (Escala horizontal, 1-2 semanas)

Acción	Qué hacer	Resultado esperado

Replicar app	2-3 réplicas de PHP app detrás de balanceador (Traefik/Nginx).	Mayor capacidad de concurrencia y resiliencia.
Estado compartido	Sesión y caché centralizadas en Redis.	Escalado sin pérdida de sesión al cambiar de nodo.
Separar colas	Colas por prioridad (alta/media/baja) y workers dedicados.	Menor impacto cruzado entre tareas.

Fase C (Observabilidad y confiabilidad, 1 semana)

Acción	Qué hacer	Resultado esperado
Métricas	Dashboards de latencia, throughput, cola, errores, CPU/RAM.	Detección temprana de cuellos.
Alertas	Alertar por p95 alto, backlog de jobs, 5xx, caídas.	Menor MTTR y menos incidentes prolongados.
Pruebas de carga	Escenarios semanales en entorno staging con datos reales anonimizados.	Validación continua de capacidad objetivo.

5. Cambios concretos recomendados

5.1 Infraestructura (Docker Compose)

- Agregar servicio Redis.
- Escalar servicio app a 2+ instancias en producción.
- Levantar al menos 2 workers de cola (ideal 3-4).
- Eliminar procesos de desarrollo (Vite dev server) del entorno productivo.

5.2 Configuración Laravel (.env)

- **APP_ENV=production**
- **APP_DEBUG=false**
- **CACHE_DRIVER=redis**
- **SESSION_DRIVER=redis**
- **QUEUE_CONNECTION=redis**
- **REDIS_HOST / REDIS_PORT / REDIS_PASSWORD** definidos.

5.3 Base de datos (índices mínimos)

- **sales:** (tenant_id, sale_date), (tenant_id, created_at)
- **sale_details:** (sale_id), (product_id)
- **products:** (tenant_id, status), (category_id)
- **user_notifications:** (user_id, read_at), (user_id, archived_at), (notification_id)
- **notifications:** (type, created_at)
- **sessions:** (user_id), (tenant_id), (last_activity)
- **jobs:** (queue, available_at), (reserved_at)

5.4 Aplicación (rápidas mejoras)

- Pagar listados pesados y evitar consultas sin límite.
- Cachear catálogos y configuraciones de baja variación.
- Precalcular reportes pesados o ejecutarlos por job asíncrono.
- Ajustar timeouts en notificaciones y retries para evitar duplicados.

6. Pruebas de carga recomendadas

Escenario 1: navegación + lectura (70%)

- Login, dashboard, listados de productos/ventas.
- Objetivo: p95 menor a 400 ms.

Escenario 2: operaciones de venta (20%)

- Crear ventas con 2-5 items, actualizar inventario, disparar eventos.
- Objetivo: p95 menor a 600 ms, 5xx menor a 1%.

Escenario 3: reportes y exportaciones (10%)

- Consultar reportes con filtros y exportar.
- Objetivo: no bloquear operaciones de venta ni colas.

7. Capacidad esperada tras implementación

- **Objetivo seguro:** 200 concurrentes estables.
- **Margen razonable:** 300-500 concurrentes según patrón de uso.
- **Riesgo principal:** reportes pesados sin aislamiento pueden reducir capacidad efectiva.

8. Plan operativo de 30 días

Semana	Entrega	Criterio de aceptación
Semana 1	Redis + drivers Laravel + ajuste inicial de workers.	Latencia baja 20-30% en staging.
Semana 2	Índices DB + tuning PHP-FPM + caché de catálogos.	Consultas críticas con menor tiempo y menos variabilidad.
Semana 3	Escalado horizontal de app + separación de colas por prioridad.	Sin degradación en pruebas de 200 concurrentes.
Semana 4	Observabilidad + alertas + prueba final de carga.	SLOs cumplidos para producción.

9. Checklist final antes de producción

- APP_DEBUG=false en producción.
- Backups automáticos de base de datos y prueba de restauración.
- Monitoreo de cola y alertas activas.
- Documentación de runbook para incidentes.
- Prueba de carga validada y firmada por equipo técnico.

Este documento define el mínimo recomendable para operar establemente con 200 usuarios concurrentes en Druvle Stock, manteniendo una ruta clara de escalabilidad.